Notes on Sessions in Flask

---

1. Introduction to Sessions

A **session** in Flask is used to store user-specific data across multiple requests.

☞ HTTP is stateless → each request is independent
☞ Sessions help maintain:

- Login state
- User data
- Temporary information

---

2. How Flask Sessions Work

- Flask stores session data in **secure cookies**
- Data is:
  - Signed using `secret_key`
  - Stored in browser
  - Protected from tampering

---

3. Creating a Session (Setup)

```
from flask import Flask, session

app = Flask(__name__)

# REQUIRED for session
app.secret_key = "mysecretkey"
```

---

4. Adding Values to Session

```
@app.route("/set")
def set_session():
    session["username"] = "siva"
    session["email"] = "siva@gmail.com"
    return "Session data stored"
```

---

5. Getting Values from Session

```
@app.route("/get")
def get_session():
    if "username" in session:
        return f"Welcome {session['username']}"
    return "No session found"
```

☞ Safe way:

```
user = session.get("username")
```

---

6. Removing Specific Value (pop)

```
@app.route("/remove")
def remove():
    session.pop("username", None)
    return "Username removed"
```

---

7. Clearing Entire Session

```
@app.route("/clear")
def clear():
    session.clear()
    return "All session data cleared"
```

---

8. Session Time (Auto Expiry)

Setting Session Timeout

```
from datetime import timedelta

app.permanent_session_lifetime = timedelta(minutes=10)
```

Making Session Permanent

```
@app.route("/login")
def login():
    session.permanent = True
    session["user"] = "siva"
```

```
    return "Session started"
```

☞ Without `session.permanent = True` → timeout will NOT work

---

9. How Session Expiry Works

- Timer resets on every request
- If user inactive → session expires
- Default → session ends when browser closes

---

10. Storing Session with Cookie Name

Default Cookie Name

```
app.config["SESSION_COOKIE_NAME"] = "session"
```

Custom Cookie Name

```
app.config["SESSION_COOKIE_NAME"] = "my_session_cookie"
```

☞ Now browser stores session data under:

```
my_session_cookie
```

---

11. Complete Example (All Concepts)

```
from flask import Flask, session, redirect, url_for, request
from datetime import timedelta

app = Flask(__name__)
app.secret_key = "secret123"

# Session time
app.permanent_session_lifetime = timedelta(minutes=5)

# Custom cookie name
app.config["SESSION_COOKIE_NAME"] = "my_session"

@app.route("/", methods=["GET", "POST"])
def login():
```

```
    if request.method == "POST":
        session.permanent = True
        session["user"] = request.form["username"]
        session["email"] = request.form["email"]
        return redirect(url_for("home"))

    return '''
        <form method="post">
            Name: <input type="text" name="username"><br>
            Email: <input type="text" name="email"><br>
            <input type="submit">
        </form>
    '''

@app.route("/home")
def home():
    if "user" in session:
        return f"Welcome {session.get('user')} ({session.get('email')})"
    return "Session expired or not logged in"

@app.route("/remove")
def remove():
    session.pop("email", None)
    return "Email removed"

@app.route("/logout")
def logout():
    session.clear()
    return "Logged out"
```

---

12. Important Methods Summary

Operation	Code
Create session	`session["key"] = value`
Get value	`session.get("key")`
Check key	`"key" in session`
Remove one	`session.pop("key")`
Clear all	`session.clear()`

---

13. Security Notes

✓ Always set `secret_key`
✓ Do NOT store passwords in session
✓ Use HTTPS in production
✓ Use:

```
app.config["SESSION_COOKIE_HTTPONLY"] = True
app.config["SESSION_COOKIE_SECURE"] = True
```

---

14. Common Errors

❌ Secret key missing

✓ Fix:

```
app.secret_key = "your_key"
```

❌ KeyError

✓ Fix:

```
session.get("user")
```

---

15. Key Interview Points

- Flask sessions use **secure cookies**
- Requires `secret_key`
- Default cookie name → `session`
- Can change using `SESSION_COOKIE_NAME`
- `session.permanent = True` enables timeout
- `session.clear()` removes all data

---

16. Real-Time Flow

1. User logs in
2. Store data in session
3. Access in protected pages
4. Timeout or logout clears session